Model Import and Scan

MVP handover documentation: install, code, package, deploy, operate

Item	Value
Version	0.1.0 (MVP, `pyproject.toml`)
Date	2 November 2026
Code location	`c:\Roshin\Kiro\Projects\model-download-scan` (not yet in Git)
Language and tools	Python 3.12 target, AWS CDK v2 (Python), Docker, AWS Fargate
AWS Region	us-east-1
Status	Code complete and tested locally (540 tests, 95.88% coverage). **Never deployed or run against a real AWS account.**

Built with the AI-DLC workflow (scope `mvp`, two Units). The design and review records are in the `aidlc/` folder of the project. Every statement here was taken from the code, the README, or those records. Section 11 lists what has not been verified.

Contents

1. What the MVP does
2. How it works (architecture and flow)
3. Where everything is (code map)
4. Inputs: the start event, the model limits, the deployment settings
5. What the scan checks
6. Prerequisites before the first deployment
7. Install and check the code on a developer machine
8. Package and deploy
9. Run it: send an event and read the results
10. Operate: alerts, logs, records, and recovery
11. What has not been verified, known limits, and open items
12. Reference: limits and names at a glance; where the design records are

1. What the MVP does

The application brings an open-weight model from the public Hugging Face site into your own Amazon S3 bucket, but only after it has been scanned for leaked secrets and personal data. It is event driven: nothing runs until an event names a model.

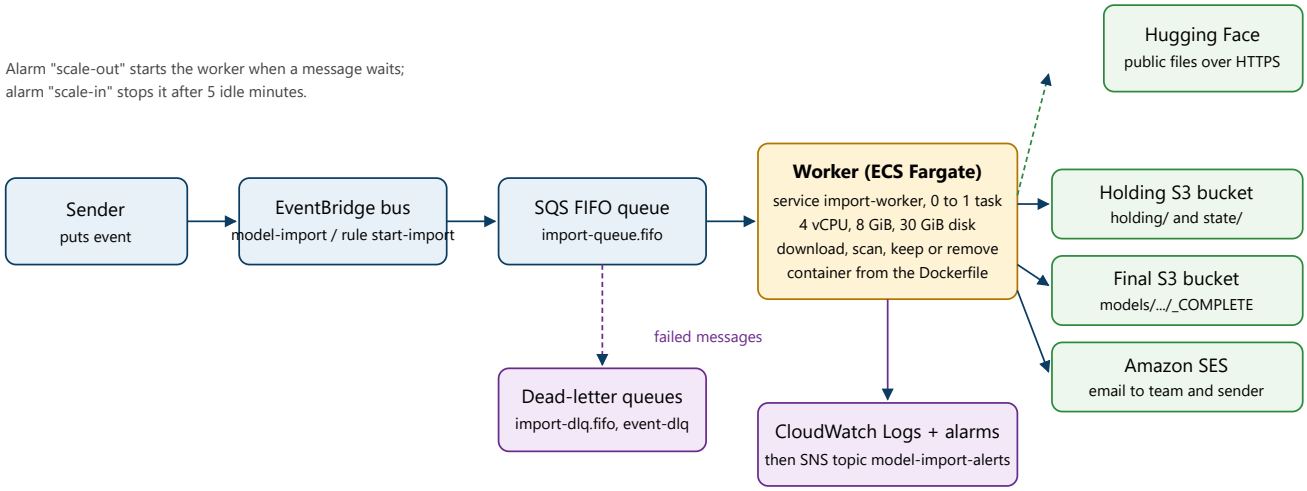
1. A person or system puts an event on the `model-import` Amazon EventBridge bus. The event names a model (`owner/name`), an exact version (a 40-character commit hash), and the sender's email address.
2. A worker running on AWS Fargate downloads every file of that version into a **holding** S3 bucket.
3. The worker scans the files. Text and settings files are searched for secrets and personal data. Weight files are checked by their file type and, for `.safetensors`, by their header.
4. A **clean** model is copied into the **final** S3 bucket. A `_COMPLETE` marker file is written last, so a model is complete only when the marker exists.
5. If a download fails, the scan finds anything, the 6-hour limit passes, or the copy fails, everything the import created is **removed** and an email goes to the team mailbox and to the sender.

Every event ends in exactly one of five outcomes: `stored`, `removed-with-email`, `removed-email-failed`, `rejected`, or `refused-already-stored` (section 10).

Scope of this first version: public models only (no Hugging Face account or token), smaller models (up to 1,000 files and 20 GB; the typical case is about 16 GB), one import at a time, one environment (`staging`) in one AWS account and Region.

> **Read this first.** The code and its automated tests are complete, but the application has **never been deployed**, the container image has never been built, and no real model, real AWS service, or real email has been used. The tests use stand-ins for S3, SQS, SES, and Hugging Face. Plan for a careful first deployment with a small model (section 8 and section 11).

2. How it works (architecture and flow)



The life of one import

1. **Start event.** The bus rule `start-import` sends events with `source = model.import` and `detail-type = ModelImportRequested` to the FIFO queue. The queue hands out one message at a time, so imports never overlap.
2. **Worker starts.** An alarm notices a waiting message and sets the Fargate service to 1 task (about 3 minutes). The worker receives the message, keeps it alive with a heartbeat every 5 minutes, and sets task protection so the platform does not stop it mid-import.
3. **Check the event.** A bad event is rejected: nothing is downloaded and the team (and the sender, if the address is well formed) gets an email that names each failed field, never its value.
4. **Front door.** If the outcome already exists, nothing is done. If a `_COMPLETE` marker for this model version already exists from another import, the event is refused (`refused-already-stored`).
5. **6-hour clock starts** when the worker takes the event off the queue (queue waiting time does not count).
6. **Download** the file list from Hugging Face, check the limits (at most 1,000 files and 20 GiB), then download up to 4 files in parallel to `holding/<importId>/`. Each download is checked for size and hash, and retried up to 3 times (waits of 5, 10, 20 seconds; at most 30 minutes of waiting in total).
7. **Scan** (section 5). Any finding fails the whole import.
8. **Keep.** Copy every file server-side to `models/<owner/name, lowercase>/<version>/`, write `_COMPLETE` last, then empty the holding folder. Once the marker is written, nothing can undo the model.
9. **Remove** (on failure). Delete the marker first (if it is this import's), then every file the import created, then list again to prove nothing is left. Anything still found is reported as a "leftover".
10. **Finish.** Email (on failure), one `import_outcome` log line, the one-time outcome object, and the final record. The message is deleted from the queue only after the outcome exists.

AWS resources created by the CDK stack

One CDK stack (default name `staging`) in one account and Region (us-east-1). Names marked "fixed" are the same in every deployment, so only one stack fits in an account and Region.

Resource	Name / setting	Purpose
EventBridge bus and rule	`model-import` (fixed), rule `start-import`; accepts events only from principals of this account	Start events
SQS FIFO queue	`import-queue.fifo`; visibility 15 min; one receive, then to the dead-letter queue; encrypted	One import at a time
Dead-letter queues	`import-dlq.fifo` (14 days), `event-dlq` (14 days)	Crashed imports; undeliverable events
VPC	New; 2 public subnets, no NAT gateway; free S3 gateway endpoint	Network for the task
ECS cluster and service	Service `import-worker`, Fargate x86_64, 4 vCPU, 8 GiB, 30 GiB disk, desired count 0 to 1	Runs the worker
Auto scaling alarms	`model-import-scale-out`, `model-import-scale-in`	Start and stop the worker
Holding bucket	Name in output `HoldingBucketName`; `holding/` expires after 2 days, `state/` after 30 days	Downloads in progress; records and outcomes
Final bucket	Name in output `FinalBucketName`; no expiry on models	Stored models

IAM roles	Task role (least privilege; no `s3:*`, no `Resource: *`), execution role	Permissions
SES configuration set	`model-import` (bounces and complaints go to the alert topic)	Failure emails
CloudWatch Logs	Log group `/model-import/worker`, 30 days	Application logs, metric filters
SNS topic and alarms	`model-import-alerts`; alarms and rules listed in section 10.2	Alerts to the team mailbox
AWS Budget	`model-import-monthly`, default 100 US dollars	Cost alert

Estimated cost in staging (from the design, not measured): the worker costs about 0.20 US dollars per hour of import; five one-hour imports a day is about 1 US dollar a day. Storage is about 0.40 US dollars a month for each 16 GB model kept. Queues, alarms, logs and SES are a few dollars a month. The estimate uses on-demand prices and should be checked on the AWS pricing page before the first deployment.

3. Where everything is (code map)

All code is in the project folder `c:\Roshin\Kiro\Projects\model-download-scan`. Git is not installed on the build machine, so there is **no repository and no history yet**: the first step for the new owner is to create one (section 6).

Top-level layout

Path	What it is
`src/model_import/`	The worker application (Python package `model_import`, 14 modules, about 4,600 lines)
`infra/`	The AWS CDK code: `config.py` (deployment settings) and `model_import_stack.py` (the whole stack)
`app.py`, `cdk.json`	CDK entry point (`python app.py`); default stack name `staging`
`tests/`	540 automated tests in `unit/`, `integration/`, `e2e/`, `infra/`
`Dockerfile`, `.dockerignore`	The container image of the worker; the build context holds only `pyproject.toml` and `src/`
`pyproject.toml`	Package metadata, exact dependency pins, Black, Ruff, pytest and coverage settings
`README.md`	Operating guide: contract, settings, deploy, outcomes, records, recovery, limits
`docs/handover/`	This document (HTML source and PDF)
`aidlc/`	AI-DLC records: requirements, stories, designs, reviews, test results, CI design (section 12)
`.venv/`, `cdk.out/`	Local only (virtual environment, CDK output); not part of the deliverable
`.gitignore`, `AGENTS.md`, `.kiro/`	Git ignore rules; AI-DLC onboarding and engine files (tooling, not application code)

The application modules (`src/model_import/`)

Module	What it does

`worker.py`	Program entry point (command `model-import-worker`). The queue loop: receive, heartbeat, task protection, 7-hour watchdog, SIGTERM handling, exit codes, builds the AWS clients with time-outs, deletes the message after the outcome exists
`event.py`	Reads and checks the start event (contract C1); classifies a message as a start event, a rejected event, or not a start event; checks file paths (including the reserved name `_COMPLETE`)
`coordinator.py`	The state machine of one import: front door, deadline checks, download, scan, keep, removal, email, the single outcome and the final record
`hf_client.py`	The only module that knows the Hugging Face web addresses: file listing, streaming download in 8 MiB chunks, speed floor, no token, certificate check
`downloader.py`	Downloads up to 4 files in parallel, enforces 1,000 files / 20 GiB, checks size and hash, uploads to the holding bucket
`retry.py`	Retry rule: 3 retries, waits 5/10/20 s, 30-minute budget, `Retry-After` up to 60 s; never waits past the deadline
`scanner.py`	The leak scanner (section 5)
`custodian.py`	All changes to the final location: marker reader, server-side copy (multipart above 5 GiB), `_COMPLETE` marker last, removal with proof by listing, leftovers
`notifier.py`	Builds and sends the emails (failure, rejection, already-stored); one email per recipient; daily limits; never includes a finding value
`store.py`	S3 access for `record.json`, the one-time `outcome.json` (conditional create), and the two daily email counters
`deadline.py`	The 6-hour `Deadline` value (injected clock) and the reason text "the time limit of 6 hours passed"
`models.py`	Data types: requests, file sets, findings, records, outcomes, leftovers; paths and the import id
`logs.py`	JSON logger with an allow-list of fields, so no secret or personal value can reach a log
`settings.py`	Reads the worker's settings from environment variables (section 4)

The tests (`tests/`)

Folder	Contents
`unit/`	444 tests, one file per module (`test_coordinator.py`, `test_custodian.py`, ...), plus `test_rule_coverage.py`, which checks that every business rule ID (BR1.1 to BR11.3) is named by some test
`integration/`	19 tests of the worker loop with stand-in queue, storage and email
`e2e/`	7 end-to-end tests: event to outcome through the real code with stand-in services
`infra/`	70 CDK template tests (permissions, bucket settings, lifecycle rules, alarms)

4. Inputs

4.1 The start event (contract C1)

```
{
  "source": "model.import",
  "detail-type": "ModelImportRequested",
  "detail": {
    "modelName": "owner/name",
    "exactVersion": "<40 characters of 0-9 and a-f>",
    "senderEmail": "person@example.com"
  }
}
```

Field	Rule
`modelName`	`owner/name` ; letters, digits, dot, underscore and hyphen only. A name with one part (no `owner/`) is rejected
`exactVersion`	The full 40-character commit hash (lower-case hex), not a branch or tag
`senderEmail`	One plain email address. It receives the failure email, so it should be a real mailbox that SES may send to

- An event that fails these checks is **rejected**: no download, an email naming the failed fields, outcome `rejected`.
- A message that is not a start event at all (wrong source or detail-type, unreadable body) gets no email and no outcome. It is logged as `event_rejected` and deleted.
- The sender needs permission to call `events:PutEvents` on the bus. The bus itself accepts only principals of the same account.

4.2 Limits on the model

Limit	Value
Files per model version	At most 1,000 (checked before any download)
Total size	At most 20 GB (counted as 20 GiB so it fits the 30 GiB task disk)
Time	6 hours per import from the moment the worker takes the event; 7-hour watchdog as a backstop
Download speed	Average below 1 MB per second over 10 minutes fails the import with "the download is too slow"
Text file for the scan	Up to 200 MB and 5 minutes per file; a line over 1 MiB is scanned in overlapping windows
Hugging Face access	Public models only; no account and no token (any `HF_TOKEN` setting is ignored on purpose)

4.3 Deployment settings (what the owner supplies)

Pass each as CDK context (`-c name=value`) or as an environment variable. A context value wins. A missing or invalid setting stops `cdk synth` and names every problem at once, without echoing the values.

Setting	Environment variable	Notes

`teamAddress`	`MODEL_IMPORT_TEAM_ADDRESS`	Team mailbox. Gets every failure email and the alerts. **Required**
`senderAddress`	`MODEL_IMPORT_SENDER_ADDRESS`	The "from" address of the emails. Must be verified in SES. **Required**
`breakGlassRoleArns`	`MODEL_IMPORT_BREAK_GLASS_ROLE_ARNS`	Comma-separated IAM role ARNs of administrators who can still read the buckets (the bucket policies deny everyone else). **Required, at least one**; the stack refuses to build without it. Form: `arn:aws:iam::<account-id>:role/<name>`
`budgetLimitUsd`	`MODEL_IMPORT_BUDGET_LIMIT_USD`	Monthly budget alert in US dollars. Default 100
`stackName`	`MODEL_IMPORT_STACK_NAME`	Letters, digits, hyphens. Default `staging` (also set in `cdk.json`)
`smokeCheckRoleArn`	`MODEL_IMPORT_SMOKE_CHECK_ROLE_ARN`	Optional. A role the pipeline's smoke check uses; it is added to the bucket-policy exemption list

4.4 The worker's own settings (set by the stack, not by you)

The CDK stack puts these into the container's environment. You only need them to run the worker by hand.

Variable	Meaning
`MODEL_IMPORT_HOLDING_BUCKET`	Holding bucket name (required)
`MODEL_IMPORT_FINAL_BUCKET`	Final bucket name (required)
`MODEL_IMPORT_QUEUE_URL`	Import queue URL (required)
`MODEL_IMPORT_SENDER_ADDRESS`, `MODEL_IMPORT_TEAM_ADDRESS`	The two mail addresses (required)
`MODEL_IMPORT_SES_CONFIGURATION_SET`	Optional SES configuration set name
`AWS_REGION`	Defaults to `us-east-1`

The worker needs **no password, key or token**. It uses the task role of AWS Fargate for AWS access.

5. What the scan checks

Any single finding fails the whole import. There is no way to accept a finding in this version. A finding records the file name, the place, and the kind. It never records the matched text, any part of it, or the text around it.

File class	Which files	What happens
Text and settings	Endings `json`, `jsonl`, `txt`, `md`, `yaml`, `yml`, `py`, `jinja`, `toml`, `cfg`, `ini`, `csv`, `tiktoken`, and files with no ending	Read in 1 MiB parts and searched with the patterns below
Weight (allowed)	`.safetensors`	Content not scanned; the header length and header are checked

Weight (not allowed)	`bin, pt, pth, pkl, pickle, ckpt, h5, onnx, gguf, msgpack, npz`	A finding "disallowed weight type" (these formats can carry executable code or cannot be read safely)
Other endings	Anything else	Kept, not scanned (an owner decision recorded in the design)

Patterns (the smallest set for the first version; a pattern may be added, none removed): cloud access keys (`AKIA` / `ASIA` keys), private key blocks, tokens (for example `hf_`, `ghp_`, `github_pat_`, `xoxb-`, `sk-`, and JSON web tokens), passwords and secrets assigned in text, email addresses, phone numbers, government-ID-style numbers (nnn-nn-nnnn), and street addresses. Up to 100 findings are recorded per file.

6. Prerequisites before the first deployment

None of these exists yet. They are the owner's to arrange.

#	Prerequisite	Notes
1	A Git repository on a hosting service, with a main branch	Install Git, run `git init`, commit. The team works on one main branch and commits small changes often. Do not commit `.venv` or `cdk.out` (already in `.gitignore`)
2	An AWS account and a way to sign in from the machine that deploys	The common AWS account in us-east-1. Use a role or SSO session, not long-lived keys
3	Tools on the deploying machine	Python 3.12+, Node.js (installed here: 24.19.0) and npm, the AWS CDK command line (`npm install -g aws-cdk`), the AWS CLI, and **Docker** (CDK builds the image at deploy time)
4	`cdk bootstrap` of the account in us-east-1	One time. Creates the asset repository and roles that the deployment uses
5	Email set up in Amazon SES (us-east-1)	Verify the sender address (`senderAddress`). While SES is in the sandbox, also verify the team mailbox and any sender addresses that will receive mail, or ask AWS for production access
6	Break-glass administrator role ARN(s)	At least one. Without one the stack will not build
7	The team mailbox that will receive alerts	After the first deployment, click the confirmation link SNS sends to it
8	A small public Hugging Face model for the first test	Within the limits in section 4.2 (small enough to download in minutes). Note its 40-character commit hash
9	A CI platform (optional for a manual first deployment)	Design: AWS CodePipeline with CodeBuild (section 11). Not built yet

7. Install and check the code on a developer machine

Python 3.12 or newer is needed. The tests on the build machine ran on Python 3.13.2; the target is 3.12, so run them once on 3.12 as well.

```
# Windows PowerShell, from the project folder
python -m venv .venv
.venv\Scripts\activate
pip install -e ".[dev]"
```

On macOS or Linux, activate with `source .venv/bin/activate`. The `dev` extra installs the test tools and the CDK library at the exact versions pinned in `pyproject.toml`.

Checks

```
python -m pytest --cov --cov-fail-under=80      # the project check; takes several minutes
black --check src infra tests app.py              # formatting
ruff check src infra tests app.py                 # lint
pip-audit --local                                 # known vulnerabilities in installed packages
```

- The pytest command is the project check. It fails below 80 percent line coverage. The tests need no AWS account and no network: S3, SQS, SES and Hugging Face are stand-ins (moto and respx). Last result: **540 passed, coverage 95.88%**.
- Black and Ruff were clean on all 44 files at the last run.
- `pip-audit` showed 12 findings, all in `pip` 24.3.1 inside the local virtual environment and none in the pinned project packages. Upgrade `pip` first and re-run.

8. Package and deploy

8.1 How the application is packaged

- **Python package:** `pip install .` installs the `model_import` package and the command `model-import-worker` (entry point `model_import.worker:main`).
- **Container image:** built from the `Dockerfile` at the project root. Base image `python:3.12-slim` (the build argument `BASE_IMAGE` can replace it), runs as the non-root user `worker`, entry point `model-import-worker`. Only `pyproject.toml` and `src/` go into the build context (`.dockerignore`).
- **You do not push the image yourself.** `cdk deploy` builds it (platform linux/amd64) and publishes it to the repository that `cdk bootstrap` created, then points the Fargate task at it. This needs Docker running on the deploying machine.

To try the image build on its own (not yet done on any machine):

```
docker build -t model-import:local .
```

> The stack builds the image without passing a base-image digest, so every build starts from the tag `python:3.12-slim` and the base image can change between builds. Pinning it by digest needs a small stack change (add `build_args` in `infra/model_import_stack.py`, function `_asset_image`).

8.2 First deployment, step by step

1. **Sign in to the AWS account** (for example `aws sso login --profile <profile>`) and select us-east-1.
2. **Install the CDK command line:** `npm install -g aws-cdk`.
3. **Bootstrap once:**

   ```
   cdk bootstrap aws://<account-id>/us-east-1
   ```

4. **Set the deployment settings.** Either in the environment:

   ```
   $env:MODEL_IMPORT_TEAM_ADDRESS = "team@example.com"
   $env:MODEL_IMPORT_SENDER_ADDRESS = "sender@example.com"
   $env:MODEL_IMPORT_BREAK_GLASS_ROLE_ARNS = "arn:aws:iam::<account-id>:role/<admin-role>"
   ```

   or as `-c` options on the commands below.
5. **Look before you deploy.** Synthesis shows the template and any setting problem:

   ```
   cdk synth
   cdk diff
   ```

6. **Deploy:**

   ```
   cdk deploy
   ```

   It prints the IAM changes for approval. The first deployment builds the image, which takes a few minutes. Keep the stack outputs: `EventBusName`, `ImportQueueUrl`, `HoldingBucketName`, `FinalBucketName`, `AlertTopicArn`.
7. **Confirm the SNS subscription:** open the email SNS sent to the team mailbox and click the link. Until then no alert reaches anyone.
8. **Check SES:** the sender identity must be verified, and while in the sandbox the recipients too. Send a test email from the console to the team mailbox.
9. **(Optional) activate the cost allocation tag** `app` once in the Billing console so the budget is filtered to this application. Until then it measures the whole account.
10. **Run the first import with a small model** (section 9) and watch the logs.

8.3 Deploying a change

1. Make the change; run the checks in section 7 (all must pass).
2. Make sure no import is running: the service desired count is 0 and the queue is empty (the deployment sets maximum healthy percent to 100, so a new task is never started beside a running one, and a running import is protected from being stopped).
3. Run `cdk diff`, then `cdk deploy`. CloudFormation rolls back a failed update by itself; the buckets are never touched by a rollback.
4. Send a test event for a small model.

> **Removing the stack.** Both buckets have a "retain" removal policy, so `cdk destroy` leaves them (and the models) in place. Delete them by hand only on purpose.

9. Run it: send an event and read the results

9.1 Send a test event

Use a small public model. The hash below is only an example: use the real 40-character commit hash of the version you want.

```
aws events put-events --region us-east-1 --entries file://event.json
```

with `event.json`:

```
[
  {
    "EventBusName": "model-import",
    "Source": "model.import",
    "DetailType": "ModelImportRequested",
    "Detail": "
{\"modelName\":\"owner/name\",\"exactVersion\":\"0123456789abcdef0123456789abcdef01234567\",\"senderEm
ail\":\"person@example.com\"}"
  }
]
```

The first task starts about 3 minutes after the event reaches the queue. A clean small model should end in `stored` within minutes; a 16 GB model is expected to take about an hour (a design target, not yet measured).

9.2 Read the result

1. **Logs:** CloudWatch log group `/model-import/worker`. Every line is one JSON object. The result line is `"event": "import_outcome"`. Example Logs Insights query for one import:

   ```
   fields @timestamp, event, status, step, outcome, reason
   | filter importId = "imp-..."
   | sort @timestamp asc
   ```

2. **The final bucket:** a stored model is at `models/<lower-case owner/name>/<exactVersion>/<path>`. It is complete only if `_COMPLETE` exists in that folder. Read only the files the marker lists.
3. **The record and the outcome** in the holding bucket under `state/<importId>/` (section 10.3).
4. **Email:** failures email the team mailbox and the sender.

The marker is JSON: `importId`, `modelName`, `exactVersion`, `files` (each with `path` and `sizeBytes`), and `completedAt`. It holds no sender address.

10. Operate: alerts, logs, records, and recovery

10.1 The five outcomes

Outcome	What happened	What it means for you
`stored`	Scan clean, every file copied, marker written	The model is in the final bucket. No email. It may carry `removalComplete: false` if the holding folder could not be fully deleted (alert b fires; remove the leftovers by hand)
`removed-with-email`	The import failed (download problem, finding, time limit, failed copy, other step). Everything was removed and every email was accepted	The sender and team were told why. Nothing should be left
`removed-email-failed`	Failed and removed, but at least one email was not accepted (SES refused it or a daily limit was reached)	Files are removed, but someone may not know. Read `emailSent` and `recipientsNotReached` in the outcome. Alert (a) fires
`rejected`	The start event had a missing or malformed field. Nothing was downloaded	An email names the failed fields (never values). Send a corrected event

`refused-already-stored`	That model version already has a `_COMPLETE` marker from another import, or a marker that cannot be read	No new import is needed. The sender and team were told

Once the `_COMPLETE` marker is written, nothing that follows can remove the model or change its outcome from `stored`.

10.2 Alerts

All alarms notify the SNS topic `model-import-alerts` (the team mailbox). Alarm names are as created in the stack.

Alert	Name	Fires when, and what to do
(a)	`model-import-a-failure-email-not-sent`	An email did not reach everyone (`emailSent = false`). Open the import's outcome; tell the people concerned by hand
(b)	`model-import-b-removal-not-complete`	Files of a failed import may still be in the buckets (`removalComplete = false`). Remove them by hand (the failure email lists up to 20 leftovers; the logs have one `leftover` line each)
(c)	rule `model-import-c-worker-stopped`	A worker task stopped with a non-zero exit code or failed to start (for example an image pull error). Look at the task's stop reason and the logs
(d)	`model-import-d-event-dead-letter-queue`, `model-import-d-import-dead-letter-queue`, `model-import-d-import-dead-letter-reminder`	A start event could not be queued, or an import stopped unexpectedly (message in `import-dlq.fifo`); the reminder repeats after 60 minutes. See 10.4
(e)	`model-import-e-email-limit-reached`	A daily email limit was reached (30 failure emails, or 20 rejection/already-stored emails, per UTC day). Find the cause (often a burst of bad events)
(f)	SES configuration set `model-import`	A bounce or complaint. Check the sender and recipient addresses
stuck	`model-import-stuck-oldest-message`	The oldest message in the queue is over 10 hours old (a message can wait up to 7 hours behind a running import). Check the worker and the scale-out alarm
cost	Budget `model-import-monthly`	Monthly spend reached the limit (default 100 US dollars)

10.3 Reading a record and an outcome

Both are JSON objects in the holding bucket under `state/<importId>/`. They are history only (nothing reads them to decide what to do next) and expire after 30 days. The `importId` is `imp-` followed by the first 32 hex characters of the SHA-256 of the event id that the bus gave the event:

```
python -c "import hashlib,sys; print('imp-' + hashlib.sha256(sys.argv[1].encode('utf-8')).hexdigest()
[:32])" <eventId>
```

Object	Key fields

`record.json` (rewritten before each step)	`status` and `step` (downloading, scanning, keeping, removing, done), `modelName`, `exactVersion`, `startedAt`, `updatedAt`, `filesCopied`, `lastKeyCopied`, `outcome`, `failureReason`, `findings` (at most 200; file name, place, kind; never values), `findingsTotal`
`outcome.json` (written once, never changed)	`outcome`, `removalComplete`, `finishedAt`, `failureReason`, `emailSent`, `recipientsNotReached` (roles `sender` / `team`, never addresses)

If the outcome object exists, the import is finished; receiving the same message again does nothing.

10.4 When an import stops unexpectedly

If the worker stops in the middle of an import (task stopped, the 7-hour watchdog, a fatal error), the import is **not cleaned up by itself**. The message moves to `import-dlq.fifo` and alerts (d) fire. A person recovers it:

1. Compute the `importId` and read `state/<importId>/record.json`. If `outcome.json` exists, nothing needs recovery.
2. If the step was `keeping` or `removing`, look in the final bucket under `models/<owner/name>/<version>/`. A folder **without** `_COMPLETE` is not a stored model; its files never expire, so remove them by hand or run the import again.
3. If the folder has a `_COMPLETE` whose `importId` is the one you are recovering, the model is stored; redriving the message finishes it as `stored` with no email.
4. Files under `holding/<importId>/` expire after 2 days; unfinished uploads are aborted after 1 day.
5. Fix the cause, then in the SQS console open `import-dlq.fifo` and choose "Start DLQ redrive". A redriven message gets a new 6-hour clock.

A failed import that ended with an email is recovered by sending the same start event again after the cause is fixed. If the email says the removal was not complete, remove the listed leftovers by hand.

10.5 Limits worth knowing

- **Email limits** (per UTC day, fixed in the code): 30 failure emails and 20 rejection/already-stored emails. Past the limit no email is sent, the event still ends with its outcome, and alert (e) fires.
- **The 6-hour limit** is checked before each step and each file, at every download chunk, and at every part of a large copy. One copy call of up to 5 GiB, the scan of one text file (5 minutes), the removal, and the email cannot be interrupted, so the design aims at the import having stopped within 10 minutes after the deadline. It is never checked after the marker is written.
- **One import at a time.** Other events wait in the queue. A second event for a model version that waits behind the first is refused once the first has stored it.
- The limits and the email limits are constants in the code, not settings.

11. What has not been verified, known limits, and open items

> **Not verified anywhere:** a deployment to AWS; the container image build (Docker is not installed on the build machine); `cdk synth` through the CDK command line (synthesis was run through `python app.py` and the CDK template tests); SES sending, the sandbox status and the verified identities; a real Hugging Face download; real timing for a 16 GB model; the copy of a file above 5 GiB in parts (tested with a stand-in because the S3 stand-in rejects the small parts needed); a Python 3.12 run (tests ran on 3.13.2); any CI pipeline.

11.1 Build and Test result

All build and test commands that could run passed (540 tests, 95.88% coverage, Black, Ruff, synthesis). Of 108 quality targets, 85 are met, 3 not met, 19 unverified, 1 not applicable (belongs to CI). The person accepted this at the Build and Test gate. The details are in `aidlc/.../construction/build-and-test/build-and-test-summary.md`.

Target	State	What it would take
Changed-module coverage of 90%: `worker.py` 89.77%, `store.py` 88.64%	Not met (project total is far above the 80% floor)	A few more tests
Stuck-queue alarm at 10 hours (design said 8)	Not met by wording; chosen on purpose, because a message can wait 7 hours behind a running import	Accept the 10 hours as the target
Task role lists whole buckets (design said by prefix)	Not met by wording; chosen on purpose so a missing object returns "not found", not "forbidden"	Accept as the target
Real import timing (16 GB in 1 hour, 20 MB/s, scan 10 min, keep 15 min), staged run of a 10 GB+ model	Unverified	A real run in the account
SES sandbox and identities	Unverified	The readiness script described in the CI design
Image build; secret scan in CI	Unverified	Docker; the CI pipeline
Tests missing for: removal beside a sibling version folder; the 100-match cap log line; 1,000-file removal; a burst of 100 bad events; durations; chunk/part constants	Unverified	About 20 more tests (offered at the Build and Test gate, not taken)

11.2 Open items for the new owner

#	Item	Notes
1	Create the Git repository and make the first commit	No history exists; the baseline for the changed Unit 1 tests was lost
2	Write the CI/CD pipeline	Design only, in `aidlc/.../construction/ci-pipeline/ci-config.md` and `quality-gates.md`: AWS CodePipeline with CodeBuild defined in CDK; runs on every push to main; gates: Black, Ruff, tests at 80%, `pip-audit`, Gitleaks, CDK template tests, Trivy image scan; then readiness check, wait for idle, deploy to staging, smoke check. The pipeline code is not in the repository
3	Decide on pinning the base image by digest	Needs a one-line stack change (section 8.1)

4	The smoke-check role	The stack only accepts its ARN (`smokeCheckRoleArn`); the role and its grants (put events, list both buckets, read the final bucket, delete the test model) would be created by the pipeline stack
5	Small wording fixes the reviewer found	(a) README recovery step 2 says "see step 4" for sending the event again; it should say step 5. (b) The failure email says the final location "may hold a complete copy" when the marker could not be checked; the files may be only partial in that case
6	Documents that disagree with the code	The NFR documents write log event names with dots (`import.outcome`); the code and the stack use underscores (`import_outcome`). Earlier design documents that need small corrections are listed in `aidlc/.../u2-import-rules/functional-design/functional-spec.md`, "Corrections to Earlier Documents"
7	A production stack	Not in the first version. The same code with another stack name; the fixed resource names mean it needs its own account or Region

11.3 Known limits of the product

- Single-name models (no `owner/`) are rejected. Any finding fails the whole import. Files with unknown endings are kept without being scanned.
- An unexpected stop is recovered by a person, not automatically.
- Only the Region us-east-1 and one stack per account are supported (fixed resource names).
- The earlier idea of a 140 GB model path was dropped: this version is for smaller models.

12. Reference

12.1 Names and numbers at a glance

Item	Value
Event source / detail type	`model.import` / `ModelImportRequested`
Bus / rule / queue / dead-letter queues	`model-import` / `start-import` / `import-queue.fifo` / `import-dlq.fifo`, `event-dlq`
ECS service / worker command	`import-worker` / `model-import-worker`
Log group / metric namespace	`/model-import/worker` (30 days) / `ModelImport`
Alert topic / SES configuration set	`model-import-alerts` / `model-import`
Cost-allocation tag	`app = model-import-scan`, `env = staging`

Holding bucket layout	`holding/<importId>/...` (2 days), `state/<importId>/record.json` and `outcome.json`, `state/counters/...` (30 days)
Final bucket layout	`models/<lower-case owner/name>/<exactVersion>/<path>` and `.../_COMPLETE`
Retries / waits / budget	3 retries per file; 5, 10, 20 s; 30 min total; `Retry-After` up to 60 s
Queue timing	Visibility 15 min, heartbeat every 5 min, watchdog 7 h, deadline 6 h
Parallelism / parts	4 files at a time; 8 MiB download chunks; 32 MiB upload parts; server-side copy in parts above 5 GiB
Email limits	30 failure emails/day, 20 rejection and already-stored emails/day (UTC day)
Pinned key dependencies	`boto3 1.43.123`, `httpx 0.28.1`; dev: `pytest 9.2.0`, `moto 5.2.5`, `respx 0.23.1`, `black 26.10.1`, `ruff 0.17.3`, `aws-cdk-lib 2.275.0`

12.2 Where the design records are

All under `aidlc/spaces/default/intents/261030-model-import-scan/`:

Folder or file	What it holds
`ideation/`	Intent statement, feasibility, scope document
`inception/`	Requirements, user stories, domain design, units, contracts (`contract-design/contract-summary.md`), delivery plan
`construction/u1-import-slice/`	Unit 1 (the end-to-end slice): functional design, NFRs, infrastructure design (`infrastructure-design/`, including `cicd-pipeline.md` and `monitoring-design.md`), `code-generation/code-summary.md`
`construction/u2-import-rules/`	Unit 2 (the rules: rejection, deadline, all-or-nothing move, five outcomes, records): the same set of documents; `functional-design/functional-spec.md` and `rules.md` (rules BR6.1 to BR11.3)
`construction/build-and-test/`	Build instructions, integration test instructions, test results, the target matrix, cross-unit traceability
`construction/ci-pipeline/`	CI design: `ci-config.md`, `quality-gates.md`
`verification/`	Phase checks (inception, construction)
`aidlc-state.md`, `audit/`	Workflow state and the decision log
`aidlc/spaces/default/memory/`	Team rules: one main branch, test-after, 80% coverage, Black and Ruff, no secrets in code or logs

12.3 Security design in brief

- No secrets: the worker needs no password, key or token; nothing sensitive is in code or logs (the logger writes only allow-listed fields; findings never include matched text).
- Hugging Face is reached over HTTPS only with certificate checks and no credentials; downloaded files are checked for size and hash.
- Both buckets are private, encrypted, TLS 1.2 or higher only, and object access is denied to everyone except the task role, the CDK roles and the break-glass roles (and the optional smoke-check role).
- The event bus accepts events only from principals of the same account; the queue accepts messages only from the bus rule.

- The task role has no `s3:*` and no `Resource: *`; the template tests check this.
- Emails never contain a matched value, and the sender's address is never written to a log, record, outcome or marker.

End of handover document. Generated from the project files on 2 November 2026. If this document and the code disagree, the code and the records under `aidlc/` are the source of truth.